

Presentation Script — *Better Features. Better Model.*

Deck: `presentation-v3.html` (9 slides) **Total estimated time:** ~8 minutes (480 s) + Q&A
Audience: mixed technical jury — assume basic familiarity with cross-validation, but no Kaggle context.

Delivery tips - Pause after every hero number — let the audience absorb it. - Treat the recipe (Screen → Decorrelate → Engineer → Validate) as a refrain. Repeat it twice across slides 2 and 9. - The deck builds to **slide 8's 0.9987**. Don't peak too early. - Use the keyboard: `←` / `→` to navigate, `F` for fullscreen.

Slide 1 · *Better features. Better model.*

On screen: Cover image — topographic terrain at twilight with an ember-orange ridge line. Two-line title in white + italic orange.

Time: ~30 s

Wildfires don't wait. When one starts, the question for an evacuation team isn't *if* it'll move — it's *which* fires will reach a populated boundary, and *when*.

Our team trained a model on **221 fires** from the WiDS Globalthon dataset to answer that question across four time horizons: **12, 24, 48, and 72 hours**.

But we'll spend most of today *not* talking about the model. We'll talk about the features. Because in our case — and we think more often than people admit — **better features made the model**. The question is how we got from 34 raw columns to a near-perfect cross-validated AUC.

[click ►]

Slide 2 · *Screen. Decorrelate. Engineer. Validate.*

On screen: Headline mantra over a horizontal funnel of four circles: **34 → 29 → 36 → 36**. Last circle filled orange.

Time: ~45 s

Our feature-selection recipe in four words: **screen**, **decorrelate**, **engineer**, **validate**.

We started with **34 raw features** after dropping IDs and target leakage.

Then we **decorrelated** — dropped 5 features that were near-perfect twins of others. That brought us to **29**.

We **engineered** 7 new features from the strongest raw signal — distance from the fire to the populated boundary. That gave us **36**.

Finally, we **validated** that 36-feature set against the alternatives on cross-validated Brier score. The engineered set won.

The next four slides walk through each step.

[click ►]

Slide 3 · *First, see what speaks to the target.* · STEP 1 SCREEN

On screen: Diverging horizontal bar chart, 34 features sorted by signed Pearson correlation with `hit_by_48h`. Orange = positive, blue = negative.

Time: ~45 s

Step one is just looking. We computed Pearson correlation of every feature with the **48-hour target** — that's the primary horizon for evacuation planning.

Two things to notice. First, *only the top ten or so features have correlations stronger than ± 0.2* . Most features are weakly correlated. That's not a death sentence — trees can still find non-linear signal — but it tells us what's loud.

Second, the strongest single feature is `dist_min_ci_0_5h` at $r = -0.466$. The negative sign matters: the closer the fire's been to the boundary in the first five hours, the more likely it hits. We'll come back to this feature — it's the workhorse of the whole model.

But correlation alone is dangerous. Two features can both correlate with the target *and with each other*, in which case keeping both is just noise. That's where step two comes in.

[click ►]

Slide 4 · *Some signals are the same signal.* · STEP 2A FIND DUPLICATES

On screen: KaTeX-rendered Pearson formula at top with range scale and `|r| > 0.85` threshold note. 8×8 lower-triangular heatmap of the fire-growth cluster, plus a "1.000" spotlight on the right.

Time: ~60 s

Pearson r measures how linearly two features co-vary. **The numerator is covariance** — how much x and y move together. **The denominator normalizes by the product of their standard deviations**, so the result lives between -1 and $+1$, regardless of scale.

We use a threshold: $|r| > 0.85$. Two features above that line carry essentially the same information — across the full matrix, **25 pairs** clear it.

The heatmap shows the fire-growth cluster — eight features that are all near-perfectly correlated with each other. Most cells light up orange.

And the brightest pair in the entire matrix isn't even on this heatmap — it's

`relative_growth_0_5h` and `area_growth_rel_0_5h` at $r = 1.000$. *Same series, different name.* The model only needs one of them.

[click ►]

Slide 5 • Why these five are dropped. • STEP 2B DECORRELATE

On screen: Five rows showing each dropped feature, its r value with the kept twin, a **REDUNDANT** / **NEAR-CONSTANT** tag, and an arrow to the kept feature. Per-horizon Brier impact strip below.

Time: ~75 s

After applying the threshold, we dropped five features. Quickly:

One. `closing_speed_m_per_h` is just -1 times `dist_change_ci_0_5h`. Same series, opposite sign. $r = -0.998$.

Two. `closing_speed_abs_m_per_h` measures absolute rate of motion — same magnitude story as area growth. $r = +0.907$.

Three. `dist_std_ci_0_5h` tracks `dist_slope_ci_0_5h` at $r = -0.943$. Slope is more interpretable. Std goes.

Four. `projected_advance_m` is correlated with `area_growth_abs` at $r = +0.855$ and it's **92% zeros**. Two reasons to drop.

Five. `dist_fit_r2_0_5h` — the R^2 of a distance-regression fit — is nearly constant across the dataset. Noise, not signal.

Five drops. The Brier impact strip at the bottom shows what we bought. At the **12-hour horizon**, **Brier dropped by 0.034** — a real improvement on the hardest, noisiest target. The 24-hour horizon got slightly worse, the longer horizons saw small wins. Net positive.

That gets us from **34 to 29 features**.

[click ►]

Slide 6 • *Then build what raw features can't see.* • STEP 3 ENGINEER

On screen: Two side-by-side rows. Each shows a **RAW feature** (`dist_min_ci_0_5h` , importance 12.77) → **formula label** → **ENGINEERED feature** with its importance and ×LIFT badge. Below each row, a small data-driven visualization.

Time: ~75 s

Step three. Engineering.

Trees can split on continuous values, so technically you don't *need* engineered features. But on 221 rows, hand-coding a meaningful threshold lets the tree split in one decision instead of ten.

We added seven engineered features. Two examples here.

`dist_close` is just `dist_min < 5000m` . A binary flag. Fire under five kilometers? Yes or no.

The data is striking. In our training set, **all 66 hits are under 5 km**. And **zero of 152 misses are**. The threshold is essentially the answer. The visualization underneath the row shows orange-hit dots clustering left of the line, gray-miss dots scattered to the right — over five orders of magnitude of distance.

That single binary flag jumped from CatBoost importance **12.77** for the raw feature to **59.20** for the threshold version. **Four-and-a-half times** more important.

Bottom row — `log_dist_min` is `log(1 + dist_min)` . Distance is right-skewed; a few faraway fires dominate the linear scale. Log compresses the tail. Modest lift on its own — about $1.1\times$ — but it teams up with `dist_close` to flatten the search space for the gradient.

Going from **29 to 36 features**.

[click ►]

Slide 7 • *Which feature set should we ship?* • STEP 4 VALIDATE

On screen: A 3-row × 4-column matrix. Rows: `34 • RAW` , `29 • + MANUAL CUT` , `36 • + ENGINEERED` . Columns: 12h / 24h / 48h / 72h. CatBoost validation Brier in each cell. Best per column highlighted in orange. Footer: `→ SHIP THE 36-FEATURE SET` .

Time: ~60 s

Step four — validation.

We trained CatBoost on each candidate feature set and scored them on validation Brier across all four horizons. Lower is better. Best per column is highlighted in orange.

Three rows: the raw 34-feature baseline, the 29-feature manually cut set, and the 36-feature engineered set.

Look at the right two columns. At the **48-hour horizon**, going from raw to engineered drops Brier from **0.010 to 0.002** — *five times better*. At the **72-hour horizon**, **seven times better**.

The shorter horizons are noisier and the differences are smaller, but the long horizons — the ones that matter most for evacuation timing — favor engineering decisively.

The 36-feature engineered set is what we ship.

[click ►]

Slide 8 • 36 features. • THE RESULT

On screen: "36 features." in white serif. Underneath, the giant ember-orange hero number **0.9987** with **48H CV AUC** tag. Below: top-5 importance bars (**dist_close 59.20** , **log_dist_min 14.33** , **dist_min 12.77** , **dist_x_area 1.69** , **dist_per_area 1.50**). Footer: **5×3 CV ensemble • CatBoost • RandomForest • XGBoost • Brier 0.0115 • 3 of top 5 features are engineered** .

Time: ~60 s

That gives us our final result.

Zero point nine-nine-eight-seven. The 48-hour cross-validated AUC.

[pause — let the number land]

We ran a **5-fold × 3-repeat** stratified cross-validation, using a soft-vote ensemble of **CatBoost, RandomForest, and XGBoost**. Brier of **0.0115**.

The top five features by importance? **Three of the five are engineered.** **dist_close** , **log_dist_min** , **dist_x_area** , and **dist_per_area** . The raw **dist_min** is in there too — but it's *third*, behind two of its own derivatives.

One caveat we want to be honest about: this is *in-distribution* cross-validation on a small dataset.

The held-out leaderboard score will almost certainly be lower. But the methodology — the recipe — is what we're confident in.

[click ►]

Slide 9 • *Three rules.* • CONCLUSION

On screen: Same topographic ember-line backdrop as slide 1. Three numbered rules in big white serif with italic orange accents on the key word. Footer: *Better features. Better model.* • 34 → 29 → 36 • AUC 0.9987 at 48h .

Time: ~30 s

So if you take three things away from this talk:

One — cut what's redundant. Two features carrying the same signal don't double the information. They double the noise.

Two — build what's missing. A well-chosen threshold or log can be more predictive than the raw column it came from. Engineering isn't optional.

Three — trust validation. Don't pick your feature set by intuition. Let the validation curve tell you when to stop.

Better features. Better model.

[pause for questions]

Q&A Prep

Anticipated questions with prepared answers:

"Why didn't you do hyperparameter tuning?"

We did, briefly — section 15 of the notebook covers a small CatBoost grid (depth 5–7, lr 0.02–0.05, L2 3–8). The differences were ~ 0.005 AUC. Feature engineering was an order of magnitude bigger than tuning, so we prioritized that.

"How do you avoid overfitting on 221 rows?"

Three ways. **One**, we use repeated stratified k-fold (5×3) so every row gets out-of-fold predictions averaged across three random seeds. **Two**, the ensemble blends three different model families with different inductive biases. **Three**, the engineering features are domain-meaningful (distance thresholds), not arbitrary polynomial expansions — they generalize because the underlying physics generalizes.

"AUC 0.9987 looks suspicious. Is the model actually that good?"

On in-distribution cross-validation, yes. On the public leaderboard, almost certainly not. The dataset is small enough that some easy cases dominate the AUC ranking. Brier (0.0115) is a more honest read of probability quality. We expect the leaderboard to land somewhere lower but in the same ballpark.

"Why drop `dist_fit_r2_0_5h` if it has high importance somewhere?"

It doesn't — its mean importance across horizons is the lowest in the data (≈ 0.04). It's nearly constant, so it adds noise without lifting any signal.

"What about features 16–34 in the correlation chart?"

Their $|r|$ is below 0.18. We kept all of them in the candidate pool because trees can find non-linear signal correlation can't see. The validation step (slide 7) confirmed they're worth keeping in aggregate.

"Could you have engineered more features?"

Probably. The seven we added were all derivatives of `dist_min` because that was the dominant signal and the gain came almost entirely from `dist_close`. Adding more transformations of weaker features rarely helps and can hurt — see the 24-hour row on slide 7 where the engineered set actually under-performs the manual-cut set.

"What would you do differently with more time?"

Two things. **One**, build a temporal cross-validation scheme that respects the chronology of fires, not just stratified random folds. **Two**, search for engineered features automatically — feature-tools or AutoML — to confirm our hand-picked threshold isn't a local optimum.

Backup pacing

If you have only **5 minutes**: - Skip slide 4 (formula details) — verbal: "Pearson r is covariance over standard deviations, ranges -1 to +1, threshold 0.85" - Compress slides 5 and 6 to 30 seconds each — name the drop count and the lift, skip per-feature reasoning - Total: ~5:30

If you have **12 minutes**: - Add 30 seconds to slide 1 explaining the WiDS competition context and the four horizons more carefully - Add 60 seconds to slide 8 walking through the per-horizon CV AUC numbers (12h: 0.9686, 24h: 0.9877, 48h: 0.9987, 72h: 1.000) - Add a "what we'd do differently" closing thought before the three rules

— *end of script* —